



Sandya Mannarswamy

Welcome to CodeSport! This month, we will discuss algorithms to support mutual exclusion, a fundamental necessity in concurrent programming.

*W*e had a question open from June, on what could cause the result of the operation $X\%Y$ to be greater than Y . Congratulations to Arivendu Bharadwaj for sending in the correct solution. Our clue to this problem was to assume that you were solving it on a hardware system where $X\%Y$ is evaluated as $X - (X/Y)*Y$.

Assume X and Y are volatile and can be changed asynchronously. The entire operation, $X - (X/Y)*Y$, is not atomic. Consider that the sub-calculation of (X/Y) has been completed, and meanwhile the value of Y in its memory location has been changed asynchronously by another thread. Now, since the variables were declared *volatile*, the compiler does not keep X and Y in registers, but reloads it every time from memory. This causes the new value of Y to be used in the calculation. Let the original values of X and Y be 12 and 3. The computed value for X/Y is 4. Now, let the new value of Y be 2. Therefore, $(X/Y)*Y$ is evaluated as 8, leading to $X\%Y$ to be evaluated as 4, which is greater than 3 (the original value of Y).

What is the software error that caused this problem? The compiler, when generating code for $X\%Y$, expands the sequence into $X - (X/Y)*Y$. It should recognise the fact that since the variables X and Y are volatile, it needs to use consistent unchanged values of X and Y throughout the modulus computation. Therefore, it should make a copy of X and Y at the beginning of the modulus computation, and use the same values for X and Y throughout the entire computation. Basically, it needs to compute the modulus atomically, so that anomalies like $X\%Y$ being greater than Y do not occur.

Our congratulations also to Vel Murugan, Vivek Ghosh and Vijai Baskar, for their solutions on the 'Locking a linked list' problem from the May 2010 column. Last month's takeaway question was for readers

to identify the latent bugs in the code snippet given below, which uses the thread-private sum variables.

```
#include <pthread.h>
pthread_t* Threads[MAX_THREADS];
int thread_private_sums[MAX_THREADS];
int Array[NUM_ELEMS];
int Sum;

void SumArray(int chunk_size)
{
    long tid = pthread_self();
    int start = tid*chunk_size;
    int stop = tid*chunk_size + chunk_size;

    for(int i = start; i < stop; i++)
    {
        thread_private_sums[tid] += Array[i];
    }
}
```

The variable *thread_private_sums* has been declared as an array of integers. Assume that *MAX_THREADS* has a value of 16. Now we have an array of 16 integers, which represents the *thread_private_sums*. Look at the access pattern of this array. Each thread reads and updates one element of the array. Since all the threads can run concurrently, the updates to the *thread_private_sums* array can occur concurrently. Do you see any issues with that?

Consider an architecture where the cache line size is 64 bytes. Assuming that *thread_private_sums* was cache-line-aligned, we have one cache line that contains all the 16 elements of the array. As each thread reads and updates one element, it leads to this cache line ping-ponging across the processor caches, since cache coherence requires that a write to a cache line containing a memory address will result in invalidating all other